

Rfice - 런타임 하우스징 에디터

입력 디스패치부터 오브젝트 선택, 편집 모드 전환, 상태 적용까지

런타임 하우스징 에디터에서 오브젝트를 선택하고, 이동/회전/색상 편집을 요청하고, 편집 결과를 배치 데이터에 반영하는 구조를 정리한 코드 샘플입니다.

Runtime Housing Editor

Input Dispatcher

State Pattern

Prop Editor Strategy

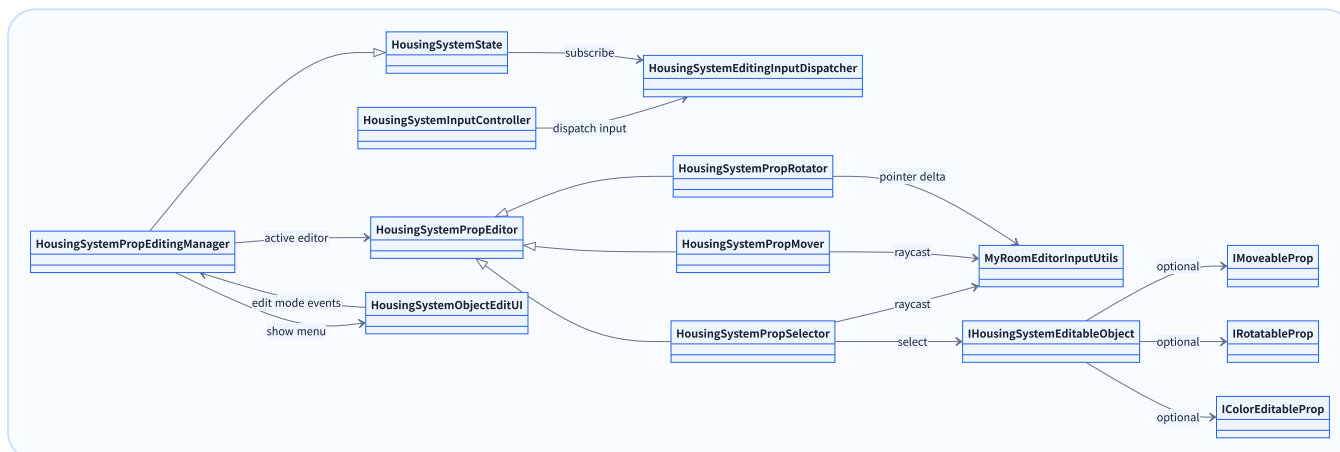
Editable Object

샘플 목표

- Input System 입력을 편집 상태 객체로 라우팅하는 구조
- 선택, 이동, 회전 편집기를 편집 모드를 교체하는 흐름
- 오브젝트별 편집 가능 기능을 인터페이스로 판별하는 방식
- UI 버튼 이벤트가 편집 모드 변경으로 연결되는 과정
- 편집 완료 시 배치 정보 갱신 이벤트 적용하는 흐름

설계 기준

- **HousingSystemInputController**: InputAction 이벤트 수신
- **HousingSystemEditingInputDispatcher**: 편집 입력 이벤트 발행
- **HousingSystemState**: 편집상태에 맞는 입력 구독
- **HousingSystemPropEditingManager**: 선택 대상과 편집 모드 관리
- **HousingSystemPropEditor**: 선택/이동/회전 편집기의 공통 동작 정의



프로젝트	Rfice
해당 영역	HousingSystem
샘플 범위	입력 디스패치, 오브젝트 선택, 편집 UI, 이동/회전 편집 모드, 배치 정보 적용
공개 방식	실제 흐름은 유지하되, 서버 저장, 리소스 로딩, 네이티브 연동, DI 세부 구현은 제외한 채용 검토용 축약
제작 영상	제작영상1 제작영상2 제작영상3

입력 수신과 편집 입력 디스패치

```
public sealed class HousingSystemInputController : MonoBehaviour
{
    [SerializeField] private HousingSystemEditingInputDispatcher editingInput;
    private HousingSystemActions actions;

    private void Awake()
    {
        actions = new HousingSystemActions();
    }

    private void OnEnable()
    {
        actions.Editor.Enable();
        actions.Editor.PointerDown.performed += OnPointerDown;
    }

    private void OnDisable()
    {
        actions.Editor.PointerDown.performed -= OnPointerDown;
        actions.Editor.Disable();
    }

    private void OnPointerDown(InputAction.CallbackContext _) => editingInput.PointerDown();
}

public sealed class HousingSystemEditingInputDispatcher : MonoBehaviour
{
    public event Action OnPointerDownEvent;
    public void PointerDown() => OnPointerDownEvent?.Invoke();
}
```

실무 의도

- InputAction 콜백에서 직접 선택/이동/회전 로직을 실행하지 않고, 편집 입력 이벤트만 발행
- 실제 처리는 활성화된 HousingSystemState와 PropEditor에서 실행
- 배치 상태와 편집 상태가 같은 입력을 서로 다른 방식으로 처리 할 수 있게 제작

포인터 Raycast와 UI 입력 필터링

```
public sealed class MyRoomEditorInputUtils : MonoBehaviour
{
    [SerializeField] private Camera raycastCamera;
    [SerializeField] private float raycastMaxDistance = 50f;
    [SerializeField] private float raycastMinDistance = 0.5f;

    private const int RaycastLimit = 10;

    public bool GetRaycastHit(out RaycastHit raycastHit, LayerMask targetLayerMask, int priorityLayer)
    {
        var ray = raycastCamera.ScreenPointToRay(PointerPosition());
        var results = new RaycastHit[RaycastLimit];
        var count = Physics.RaycastNonAlloc(ray, results, raycastMaxDistance, targetLayerMask);

        if (count == 0)
        {
            raycastHit = default;
            return false;
        }

        results = results
            .Where(hit => hit.collider != null && hit.distance > raycastMinDistance)
            .OrderBy(hit => hit.distance)
            .ToArray();

        if (results.Length == 0)
        {
            raycastHit = default;
            return false;
        }

        var priorityHits = results
            .Where(hit => hit.transform.gameObject.layer == priorityLayer)
            .ToArray();

        raycastHit = priorityHits.Length > 0 ? priorityHits[0] : results[0];
        return true;
    }

    public bool IsPointerOnUI()
    {
        var pointerEvent = new PointerEventData(EventSystem.current);
        pointerEvent.position = PointerPosition();

        var results = new List<RaycastResult>();
        foreach (var raycaster in FindObjectsOfType<GraphicRaycaster>())
            raycaster.Raycast(pointerEvent, results);

        return results.Count > 0;
    }

    public Vector2 PointerDelta()
    {
        return Mouse.current.delta.ReadValue();
    }

    private Vector2 PointerPosition()
    {
        return Mouse.current.position.ReadValue();
    }
}
```

실무 의도

- 각 편집기는 필요한 결과만 받아 선택, 이동, 회전 로직에 집중
- InputUtils는 UI 위 클릭을 먼저 걸러내고, Raycast 결과를 거리와 우선순위 레이어 기준으로 정리
- 회전 편집에서는 PointerDelta를 통해 프레임별 회전량 측정

현재 편집 상태에서만 입력 구독

```
public abstract class HousingSystemState : MonoBehaviour
{
    [SerializeField] protected HousingSystemEditingInputDispatcher input;

    protected abstract HousingSystemStateEnum TargetState { get; }

    protected abstract void Enable();
    protected abstract void Disable();
    protected abstract void OnPointerDown();

    public void OnChangedState(HousingSystemStateEnum state)
    {
        if (state == TargetState)
        {
            SubscribeInput();
            Enable();
            return;
        }

        UnsubscribeInput();
        Disable();
    }

    private void SubscribeInput()
    {
        input.OnPointerDownEvent += OnPointerDown;
    }

    private void UnsubscribeInput()
    {
        input.OnPointerDownEvent -= OnPointerDown;
    }
}
```

실무 의도

- HousingSystemState는 상태가 활성화될 때만 편집 입력을 구독하고, 다른 상태로 전환되면 입력 구독을 해제
- 실무에서는 배치모드, 편집모드가 각각 토글되어 활성화 되었을 때만 입력 구독

편집 가능 오브젝트 정의

```
public interface IMyRoomEditorEditableObject
{
    string PropNameKey { get; }

    void Focused();
    void Unfocused();
    void Selected();
    void Deselected();
    void Delete();

    bool IsMovableProp(out IMoveableProp prop);
    bool IsRotatableProp(out IRotatableProp prop);
    bool IsEditableColor(out IColorEditableProp prop);

    MyRoomPlaceProp GetPlacePropInfo();
    string GetPlacePropId();
}

public interface IMoveableProp
{
    SpawnablePropBase PropBaseComponent { get; }

    Vector3 GetPosition();
    bool IsPlaceableArea(Vector3 hitPosition, IPlaceableArea area);
}

public interface IRotatableProp
{
    Quaternion GetRotation();
    void SetRotation(Quaternion rotation);
}

public interface IColorEditableProp
{
    IReadOnlyList<MyRoomPropColor> GetColorList();
    void ChangeColor(int colorIndex);
}
```

실무 의도

- 편집 매니저와 편집기는 선택된 오브젝트가 어떤 편집 기능을 지원하는지만 확인
- 바닥/벽/소품처럼 기능이 다른 오브젝트도 같은 선택 흐름에 태우고, 이동 가능하면 Mover, 회전 가능하면 Rotator, 색상 변경 가능하면 Color UI를 활성화하는 구조

선택 에디터와 편집 대상 확정

```
public sealed class HousingSystemPropSelector : HousingSystemPropEditor
{
    public event Action<IHousingSystemEditableObject> OnObjectSelect;
    public event Action OnReleaseSelect;

    private IHousingSystemEditableObject focusedObject;

    private IEnumerator CoDetectObject()
    {
        while (true)
        {
            yield return null;

            if (!InputUtils.GetRaycastHit(out var hit, targetLayer, firstPriorityLayer))
                continue;

            if (InputUtils.IsPointerOnUI())
                continue;

            if (!hit.transform.TryGetComponent(out IHousingSystemEditableObject editable))
                continue;

            if (editable.Equals(focusedObject))
                continue;

            focusedObject?.Unfocused();
            focusedObject = editable;
            focusedObject.Focused();
        }
    }

    public override void OnPointerDown()
    {
        if (InputUtils.IsPointerOnUI())
            return;

        if (focusedObject == null || focusedObject.IsPlacementArea(out _))
        {
            ReleaseSelect();
            return;
        }

        SelectedObject?.Deselected();
        SelectedObject = focusedObject;
        SelectedObject.Selected();

        OnObjectSelect?.Invoke(SelectedObject);
    }

    private void ReleaseSelect()
    {
        SelectedObject?.Deselected();
        SelectedObject = null;
        focusedObject = null;
        OnReleaseSelect?.Invoke();
    }
}
```

실무 의도

- PropSelector는 포커스와 선택 상태를 관리하고, 선택 결과만 이벤트로 매니저에 전달

편집 매니저의 모드 전환

```
public sealed class HousingSystemPropEditingManager : HousingSystemState
{
    [SerializeField] private HousingSystemPropSelector propSelector;
    [SerializeField] private HousingSystemPropMover propMover;
    [SerializeField] private HousingSystemPropRotator propRotator;
    [SerializeField] private HousingSystemObjectEditUI objectEditUI;

    private IHousingSystemEditableObject editingObject;
    private HousingSystemPropEditor propEditor;
    private PropEditMode editingMode;

    protected override HousingSystemStateEnum TargetState => HousingSystemStateEnum.Editing;

    protected override void Enable()
    {
        propSelector.OnObjectSelect += OnObjectSelected;
        propSelector.OnReleaseSelect += OnReleaseSelected;

        objectEditUI.OnClickMoveEvent += () => OnChangedEditMode(PropEditMode.Move);
        objectEditUI.OnClickRotateFreeEvent += () => OnChangedEditMode(PropEditMode.Rotate);

        propMover.OnMoveConfirmed += OnMoveConfirmed;
        propRotator.OnRotateConfirmed += OnRotateConfirmed;

        OnChangedEditMode(PropEditMode.Select);
    }

    private void OnObjectSelected(IHousingSystemEditableObject selectedObject)
    {
        editingObject = selectedObject;
        objectEditUI.ShowSelectedPropMenu(editingObject);
    }

    private void OnChangedEditMode(PropEditMode mode)
    {
        editingMode = mode;

        propEditor?.Disable();
        propEditor = SwitchPropEditor();

        if (propEditor != null && propEditor.Setup(editingObject))
            propEditor.Enable();
    }

    private HousingSystemPropEditor SwitchPropEditor()
    {
        return editingMode switch
        {
            PropEditMode.Select => propSelector,
            PropEditMode.Move => propMover,
            PropEditMode.Rotate => propRotator,
            _ => null
        };
    }
}
```

실무 의도

- 매니저는 선택된 오브젝트 및 현재 편집 모드 관리
- 실제 선택, 이동, 회전 로직은 각각의 PropEditor가 수행하고, UI 버튼 이벤트는 PropEditMode 변경으로 연결

코루틴 기반 이동 편집

```
public sealed class HousingSystemPropMover : HousingSystemPropEditor
{
    public event Action OnMoveConfirmed;

    private IMoveableProp moveableProp;
    private bool isPlaceable;

    public override bool Setup(IHousingSystemEditableObject target)
    {
        if (!target.IsMovableProp(out moveableProp))
            return false;
        SelectedObject = target;
        return true;
    }

    public override void Enable()
    {
        StartCoroutine(CoDetectMove());
    }

    private IEnumerator CoDetectMove()
    {
        while (true)
        {
            isPlaceable = IsPlaceablePoint();
            yield return null;
        }
    }

    private bool IsPlaceablePoint()
    {
        if (!InputUtils.RaycastHit(out var hit, targetLayer, moveableProp.PropBaseComponent))
            return false;
        return hit.transform.TryGetComponent(out IPlaceableArea area)
            && moveableProp.IsPlaceableArea(hit.point, area);
    }

    public override void OnPointerDown()
    {
        if (!InputUtils.IsPointerOnUI() && isPlaceable)
            OnMoveConfirmed?.Invoke();
    }
}
```

실무 의도

- 이동 모드가 활성화된 동안 코루틴으로 Raycast 결과를 계속 추적
- IsPlaceableArea() 안에서 후보 위치로 오브젝트를 이동시키고, 배치 가능 여부를 함께 검증
- 유효한 위치에서 클릭하면 현재 위치를 확정하고 매니저에 완료 이벤트 전달

코루틴 기반 회전 편집

```
public sealed class HousingSystemPropRotator : HousingSystemPropEditor
{
    public event Action OnRotateConfirmed;

    private IRotatableProp rotatableProp;

    public override bool Setup(IHousingSystemEditableObject target)
    {
        if (!target.IsRotatableProp(out rotatableProp))
            return false;
        SelectedObject = target;
        return true;
    }

    public override void Enable()
    {
        StartCoroutine(CoDetectRotate());
    }

    private IEnumerator CoDetectRotate()
    {
        while (true)
        {
            var delta = InputUtils.PointerDelta().x;
            var rotation = rotatableProp.GetRotation() * Quaternion.Euler(0f, -delta, 0f);
            rotatableProp.SetRotation(rotation);
            yield return null;
        }
    }

    public override void OnPointerDown()
    {
        if (InputUtils.IsPointerOnUI())
            return;
        OnRotateConfirmed?.Invoke();
    }
}
```

실무 의도

- 회전 모드가 활성화된 동안 코루틴으로 PointerDelta를 계속 추적
- 포인터 이동량을 Y축 회전값으로 변환해 선택 오브젝트에 실시간 반영
- 사용자가 클릭하면 현재 회전값을 확정하고 매니저에 완료 이벤트를 전달

편집 완료 후 배치 정보 갱신

```
public sealed class HousingSystemPropEditingManager : HousingSystemState
{
    private IHousingSystemEditableObject editingObject;

    private void OnMoveConfirmed()
    {
        PublishEditedPropPlacementInfo();
        OnChangedEditMode(PropEditMode.Select);
    }

    private void OnRotateConfirmed()
    {
        PublishEditedPropPlacementInfo();
        OnChangedEditMode(PropEditMode.Select);
    }

    private void ApplySelectedColor(int colorIndex)
    {
        if (editingObject == null)
            return;

        if (!editingObject.IsEditableColor(out var colorEditable))
            return;

        colorEditable.ChangeColor(colorIndex);
        PublishEditedPropPlacementInfo();
    }

    private void PublishEditedPropPlacementInfo()
    {
        if (editingObject == null)
            return;

        var propInfo = editingObject.GetPlacePropInfo();
        placementPresenter.OnUpdatedPlacementProp(propInfo);
    }
}
```

실무 의도

- 이동/회전 편집기는 완료 이벤트만 발생시키고, 실제 배치 정보 갱신은 HousingSystemPropEditingManager가 담당
- 편집 도구별 실행 로직과 데이터 반영 지점을 분리

내용 정리

- InputAction 콜백과 편집 실행 로직을 분리
- 상태 패턴으로 배치/편집 상태의 입력 충돌 방지
- 선택, 이동, 회전을 PropEditor 전략으로 분리
- 오브젝트별 편집 가능 기능을 인터페이스로 판별
- 편집 완료 후 배치 정보 갱신 지점을 매니저로 집중